Revisitable Memory Agents with Graph-Based Memory Traversal

for Multi-Hop Long-Context Reasoning

CECS 551 Term Project – Phase 1 Proposal

Team 8: Utkarsh Balu Lubal | Kashif Manzer | Mayank Maurya | Dipak Yadav

1 Industry, Problem Definition, and Customer Pain Points

Industry/Domain: Retrieval-Augmented Generation (RAG) for long-context, multi-document question answering across domains: enterprise knowledge bases (policies, wikis, PRDs, SOPs), legal/compliance archives, customer support knowledge bases, research assistants, and education content.

Problem: Many real-world questions require **multi-hop reasoning** where the needed evidence is not located in a single "most similar" passage. Standard RAG retrieves top-k chunks (BM25/embeddings) and generates an answer; this **flat retrieval** fails when:

- evidence is distributed across multiple documents or time periods (e.g., policy update + later exception),
- relevant passages are indirectly connected (shared entities, cross-references, prior decisions),
- the query is underspecified and requires iterative refinement (discover prerequisites first).

Increasing context windows is costly and still may miss key prerequisites, increasing hallucination risk and lowering user confidence.

Example (multi-hop failure): "What decision did we make about data retention, and which compliance clause justified it?" One chunk may mention the decision; another chunk contains the justification. Top-k similarity often retrieves only one side of the chain.

Target users (general RAG):

- Companies deploying internal RAG assistants for structured recall of documentation with high confidence.
- Knowledge workers in compliance/legal/security/finance/operations who need auditable evidence trails.
- Support/enablement teams (help desks, customer support, sales engineering) navigating playbooks and KB articles.
- Students/researchers navigating course materials and paper collections (a strong demo domain).

Customer pain points:

- **Fragmented recall:** isolated chunks retrieved instead of connected evidence chains.
- **Low confidence:** weak provenance makes it hard to verify why an answer is correct.
- **Multi-hop gaps:** answers require combining 2–4 supporting facts across sources.
- **Scale friction:** retrieval/debugging gets harder as corpora grow without structure.

2 AI Application Plan (What we will build)

Goal: Build and evaluate an LLM agent with **revisitable memory callbacks** (look back multiple times during reasoning) and a **graph-structured memory** that retrieves evidence via bounded graph traversal rather than flat top-k retrieval.

Memory Graph Representation

We model memory as a **sparse graph** rather than a list:

- **Nodes:** text chunks/notes + metadata (doc, section/page/slide, timestamp/index) + extracted entities/keywords + optional short summary.
- **Edges:** NEXT (adjacency), SHARED_ENTITY (entity overlap), LEXICAL_SIM (BM25/TF-IDF similarity), optional RELATION (lightweight links).

Edge	Purpose
NEXT	Keeps local continuity; supports "read around" behavior
SHARED_ENTITY	Connects distant but concept-linked passages (people, systems, terms)
LEXICAL_SIM	Bridges wording overlap; complements entity links
RELATION (opt.)	Direct "A depends on B" / "A justified by B" style links (lightweight)

Two-Stage Retrieval + Bounded Traversal

Stage 1 — Seed selection: indexed retrieval (BM25 + entity lookup) selects a small set of promising starting nodes.

Stage 2 — Traversal retrieval: bounded beam search / priority-queue BFS expands from seeds to assemble a compact evidence subgraph capturing multi-hop chains (e.g., definition → constraint → decision → consequence). Each step scores candidates using a weighted mix of:

$$score(v) = \alpha\, Rel(q, v) + \beta\, w(e) + \gamma\, Novelty(v)$$

where $Rel(q, v)$ is query relevance, $w(e)$ is edge weight, and Novelty reduces redundancy.

Revisitable callbacks: the agent answers in cycles: draft partial answer → detect missing prerequisite → refine query → traverse again → finalize with citations and a short retrieval-path explanation.

Scalability Advantage (Why this is efficient)

Naive all-pairs similarity linking across n chunks can be $O(n^2)$. Our approach avoids all-pairs comparisons by using: **(i) indexed seed retrieval** (inverted index/entity lookup) and **(ii) sparse edge construction** with bounded degree (e.g., top-m links per node), then **(iii) budgeted traversal** at query time. In practice this yields approximately $O(n \log n)$-style scalable indexing/sparse construction behavior, with controlled query-time exploration under fixed depth/beam budgets.

Implementation and Evaluation Plan (How we validate)

Implementation: Python; chunking + entity extraction (spaCy) + keyword extraction (TF-IDF); weighted edge construction; traversal with tunable depth/beam/budget; context assembly selects top M nodes/summaries within a prompt budget and attaches path metadata.

Evaluation:

- **Benchmarks:** HotpotQA and/or 2WikiMultiHopQA (multi-hop reasoning). We will also demo on a small internal corpus (course PDFs or wiki pages).
- **Baselines:** BM25 top-k + LLM; optional embedding top-k + LLM (no traversal); traversal without callbacks (single pass).
- **Metrics:** EM/F1; evidence coverage proxy (gold facts present in retrieved context); token usage and latency.
- **Ablations:** remove edge types (NEXT only vs entities only vs lexical only); vary traversal budgets (depth/beam); callbacks vs no callbacks.

3 Why It Matters (Societal and Industry Impact)

- **General RAG improvement:** structured traversal supports multi-hop reasoning beyond similarity top-k.
- **Enterprise trust:** retrieval paths improve transparency, auditability, and high-confidence document recall.
- **Reliability:** better evidence coverage reduces hallucinations and unsupported answers.
- **Scalability:** avoids quadratic growth from all-pairs comparisons as corpora expand.

4 Research Questions

1. Does graph traversal retrieval improve multi-hop QA performance compared to flat BM25/embedding top-k retrieval?
2. Which edge types contribute most (NEXT vs entity overlap vs lexical similarity)?
3. What traversal budget (depth/beam) best balances accuracy vs token/latency cost?
4. Do revisitable callbacks improve evidence coverage and reduce hallucinations vs single-pass retrieval?

5 Expected Deliverables

- Working agent supporting: revisitable callbacks, graph-based traversal retrieval, and evidence + retrieval-path explanations.
- Results package: baseline vs traversal comparisons, ablations (edge types/budgets), and accuracy–cost tradeoff summary.
- Reproducible repository: scripts/configs for graph build → retrieval → evaluation, plus a small demo corpus pipeline.